INDEX

Name Navinavardhini .M Subject

Standard CSD Section 3rd year Roll No. 18

S.No.	Date	Title	Page No.	Teacher's Sign
1.	21/7/26	Bresenham Line Algorithm	1	
2.	21/7/26	Bresenham circle Alg	3	
3.	28/7/26	Bresenham Ellipse Alg	5	
4.	4/8/26	object with Line Alg	7	
5.	4/8/26	Implementation of DDA	9	
6.	4/8/26	Draw a Rectangle	11	
7.	4/8/26	Draw a traffic signal	13	
8.	11/8	create a smiley face	15	
9.	11/8	create olympic logo	17	
10.	4/8	create a Butterfly	19	
11.	14/9	Scan conversion of polygon	21	
12.	14/9	2D shape Transformations	25	
13.	29/9	Polygon clipping	27	
14.	29/9	cube 3D Transformation	29	
15.	29/9	cone 3D Transformation	31	
16.	6/10	create a multimedia application	33	
17.	6/10	capture video from webcam	35	
18)	4/10	create application for multimedia	37	
19)	11/10	create 3D model using Blender	38	

Completed

Exp:1 Bresenham Line Algorithm

Date:
21/7/26

Aim: To draw a line btw a point (x_0, y_0) and (x_n, y_n) using Bresenham Line Algorithm.

```
import matplotlib.pyplot as plt
def bresenham(x0, y0, xn, yn):
    dx = xn - y0
    dy = yn - y0
    p = 2*dy - dx
    x = x0
    y = y0
    x_points = [x]
    y_points = [y]
    while x < xn:
        x+1 = 1
        if p<0:
            p = p+2*dy
        else:
            y+=1
            p = p+2*dy-2*dx
        x_points.append(x)
        y_points.append(y)
    return x_points, y_points
x0 = int(input("Enter x0: "))
y0 = int(input("Enter y0: "))
xn = int(input("Enter xn: "))
yn = int(input("Enter yn: "))
```

Enter xo: 5

Enter yo: 2

Enter xn: 10

Enter yn: 7

Points on the line:

(5, 2)

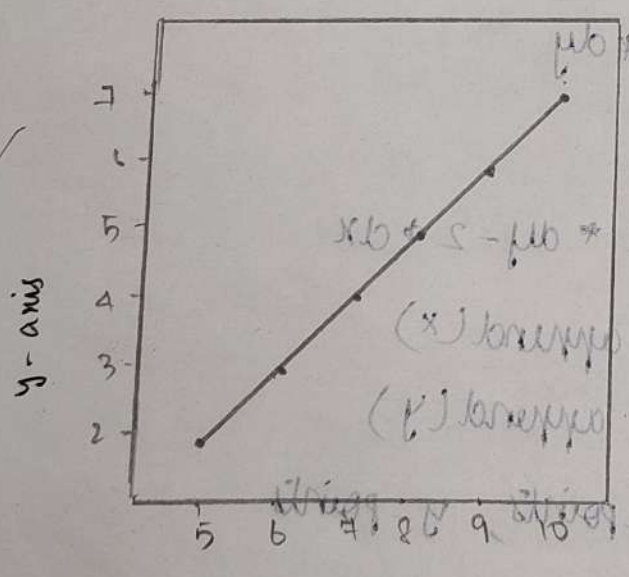
(6, 3)

(7, 4)

(8, 5)

(9, 6)

(10, 7)



o/p Verified
21/7/20

```
x_points, y_points = bresenham(x0, y0, xn, yn)
print("points on the lines:")
for (x, y in zip(x_points, y_points):
    print(f"({x}, {y})")
plt.figure(figsize=(6,6))
plt.plot(x_points, y_points, marker='s',
linestyle='-', color='blue')
plt.title('-', color='blue')
Algorithm (m<1)")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.grid(True)
plt.axis("equal")
plt.show()
```

Result:

The program displays a continuous line on a matplotlib grid display.

Bresenham Ellipse Drawing Algorithm

Aim:

To generate a ellipse with centre point (x_c, y_c) with given horizontal (r_x) and vertical (r_y) radii by calculating points for one quadrant and using 4-way symmetry to mirror them.

Code:

```
import matplotlib.pyplot as plt
def draw_ellipse(xc, yc, rx, ry):
    x_points = []
    y_points = []
    def add_symmetry_points(x, y):
        x_points.extend([xc+x, xc-x, xc+x, xc-x])
        y_points.extend([yc+y, yc+y, yc-y, yc-y])

    x = 0
    y = ry
    p1 = (ry**2) - (rx**2 * ry) + (0.25 * rx**2)
    dx = 2 * ry**2 * x
    dy = 2 * rx**2 * y
    while dx < dy:
        add_symmetry_points(x, y)
        x+1 = 1
        dx += 2 * symmetry**2
```

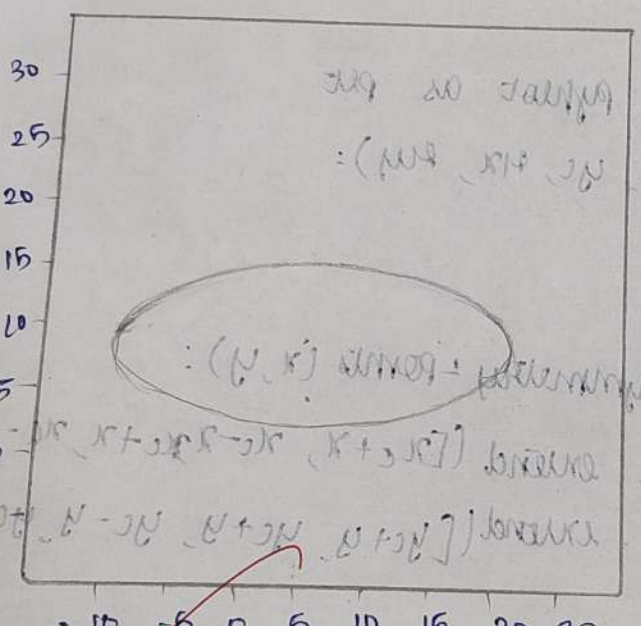
Enter center x (xc) : 5

Enter center y (yc) : 10

Enter Horizontal Radius (rx) : 15

Enter Vertical Radius (ry) : 6.

Bresenham Ellips Algorithm



s/e verified

```
        if p2 > 0:
            p2 += rx ** 2
        else:
            x += 1
            dx += 2 * ry ** 2
            p2 += rx ** 2 + dx - dy
        return x_points, y_points

xc = int(input("Enter centre x(xc): "))
yc = int(input("Enter centre y(yc): "))
rx = int(input("Enter H Radius (rx): "))
ry = int(input("Enter V Radius (ry): "))

x_pts, y_pts = draw_ellipse(xc, yc, rx, ry)
plt.figure(figsize=(6,6))
plt.plot(x_pts, y_pts, 'ro', markersize=4)
plt.title("Bresenham Ellipse Algorithm")
plt.xlabel("x_axis")
plt.ylabel("y_axis")
plt.grid(True)
plt.axis("equal")
plt.show()
```

Result:

Circle was drawn on the plot using Bresenham algorithm.

Bresenham Circle Drawing Algorithm

Aim:

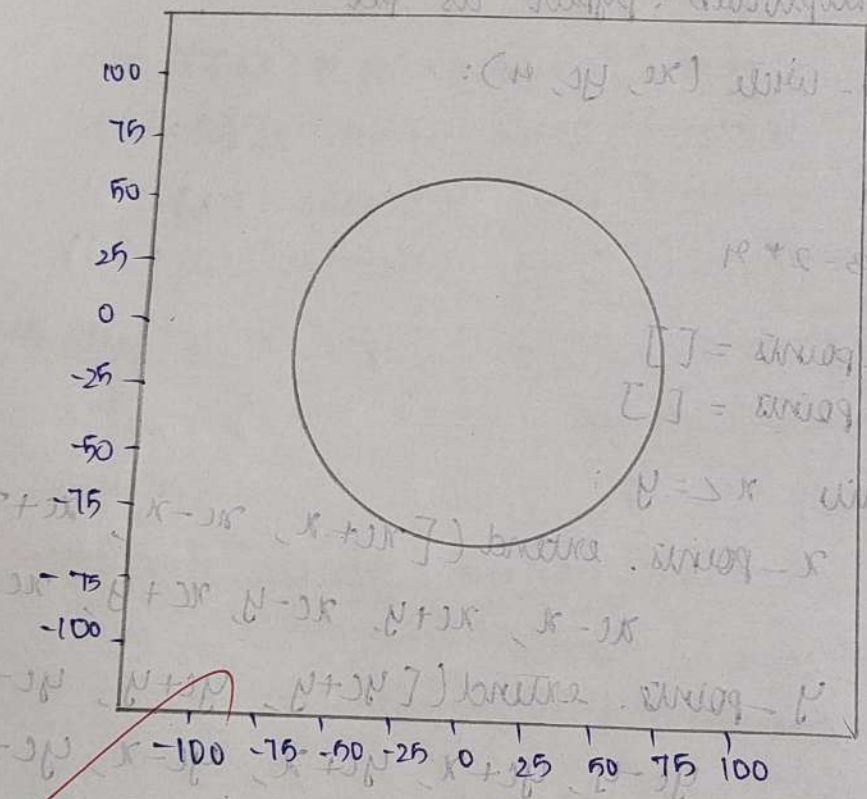
To generate a circle using Bresenham algorithm.

Code:

```
import matplotlib.pyplot as plt
def draw_circle(xc, yc, r):
    x=0
    y=r
    P=3-2*r

    x_points = []
    y_points = []
    while x<=y:
        x_points.extend([xc+x, xc-x, xc+x,
                xc-x, xc+y, xc-y, xc+y, xc-y])
        y_points.extend([yc+y, yc+y, yc-y,
                yc-y, yc+x, yc+x, yc-x, yc-x])
        if P<0:
            P=P+4*x+6
        else:
            y=y-1
            P=P+4*(x-y)+10
        x+=1
    return x_points, y_points
```

Bresenham circle algorithm



xc = int

yc = int

r = int

x_pts,

Plt. fig

Plt. ne

Plt. tit

Plt. x

Plt. y

Plt. gr

Plt.

Plt.

Result:

circle

```
xc = int(input("Enter center x (xc): "))
yc = int(input("Enter center y (yc): "))
r = int(input("Enter radius (r): "))

x_pts, y_pts = draw_circle(xc, yc, r)
plt.figure(figsize=(6,6))
plt.plot(x_pts, y_pts, 'ro', markersize=4)
plt.title("Bresenham circle algorithm")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.grid(True)
plt.axis("equal")
plt.show().
```

Result: 28/7/26

The program displays a continuous circle on matplotlib grid.

4/8/26
Exp: 4

creating an object using Bresenham's Line Algorithm

Aim:

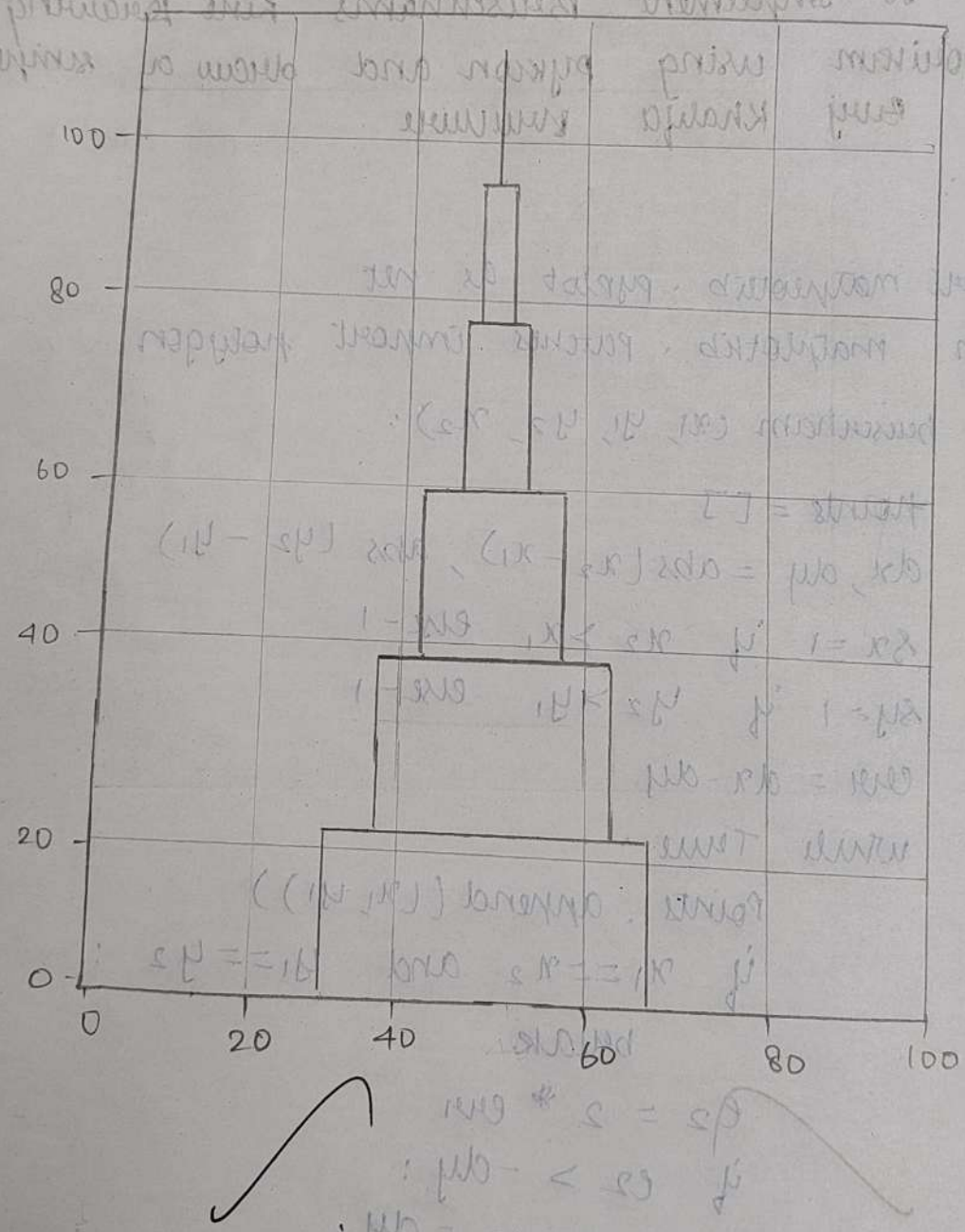
To Implement Bresenham's line drawing Algorithm using Python and draw a simple Suraj Khalifa building.

code:

```
import matplotlib.pyplot as plt
from matplotlib.patches import polygon

def bresenham(x1, y1, y2, x2):
    points = []
    dx, dy = abs(x2 - x1), abs(y2 - y1)
    sx = 1 if x2 > x1 else -1
    sy = 1 if y2 > y1 else -1
    err = dx - dy
    while True:
        Points.append((x1, y1))
        if x1 == x2 and y1 == y2:
            break
        e2 = 2 * err
        if e2 > -dy:
            err -= dy
            x1 += sx
        if e2 < dx:
            err += dx
            y1 += sy
```

Buij Kalija using Bresenham's Line Algorithm.



```
    return points
def draw_line(x1, y1, x2, y2):
    pts = bresenham(x1, y1, x2, y2)
    x = [p[0]
def line(a, b):
    p = bresenham(*a, *b)
    plt.scatter(*zip(*p), s=5, c="black")
fig, ax = plt.subplots(figsize=(5,7))
ax.set_facecolor("skyblue")
ax.add_patch(polygon([[0,0), (100,0), [100,8),
                      (0,8)], color="green"))
sections = [[(35,8), (65,8), [65,25), (35,25)],
            [(38,25), (62,25), (62,42), (38,42)]
            [(41,42), (59,42), (59,58), (41,58)]
            [(44,58), (56,58), (56,73), (44,73)]
            [(49,90), (51,90), (51,100), (49,100)]

for s in sections:
    ax.set_aspect("equal")
    ax.grid()
    plt.title("Burj Khalifa")
    plt.show()
```

Result:

Thus an object was created using Bresenham's line Algorithm

4/8/26
Exp: 5

Implementation of DDA using Bresenham's Line Algorithm.

Aim:

To implement the DDA (Digital Differential Analyzer) line drawing algorithm to draw a straight line.

Code:

```
import matplotlib.pyplot as plt
x1 = int(input("Enter x1: "))
y1 = int(input("Enter y1: "))
x2 = int(input("Enter x2: "))
y2 = int(input("Enter y2: "))
dx = x2 - x1
dy = y2 - y1
steps = max(abs(dx), abs(dy))
x_inc = dx / steps
y_inc = dy / steps

x = x1
y = y1
x_points = []
y_points = []
for i in range(steps + 1):
```

4/8/26
Exp: 5 Impl

Bres

Aim:

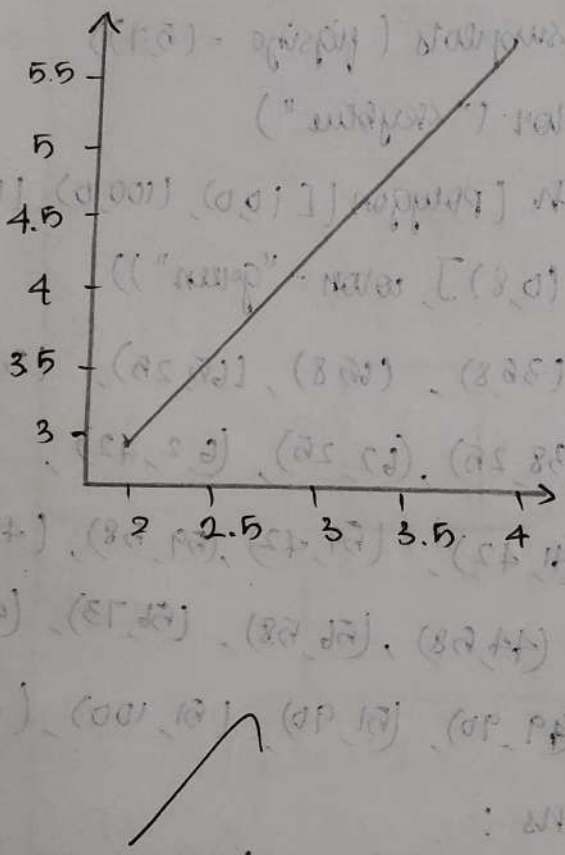
To impl
Analyzer)
a

code:

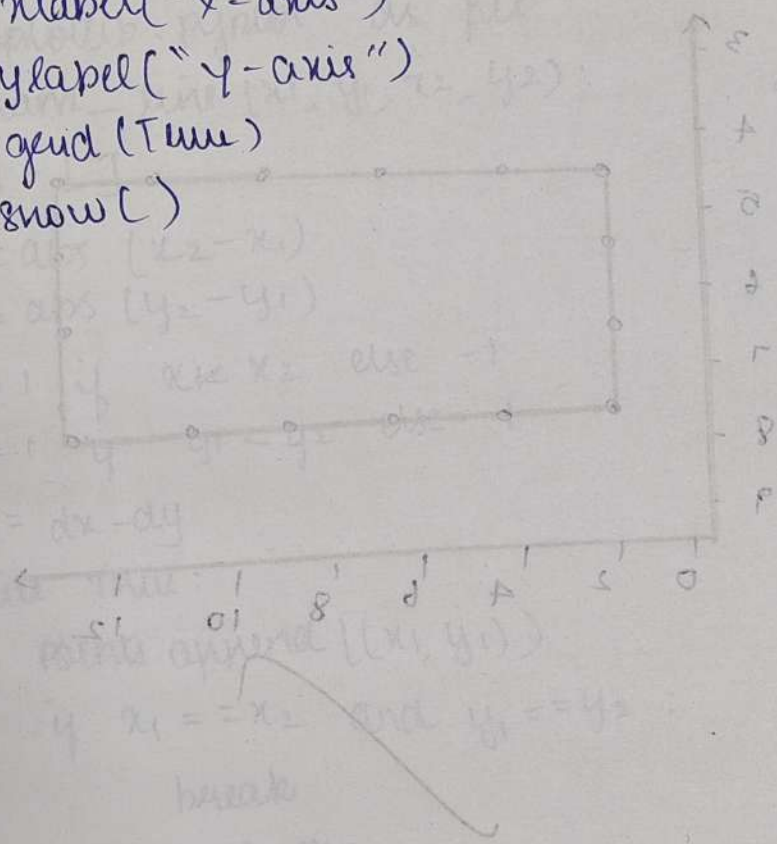
```
import matp
x1 = int (
y1 = int (
x2 = int (
y2 = int (
dx = x2 -
dy = y2 -
steps = m
x_inc = d
y_inc =

x = x1
y = y1
x_points
y_points

for i i
```

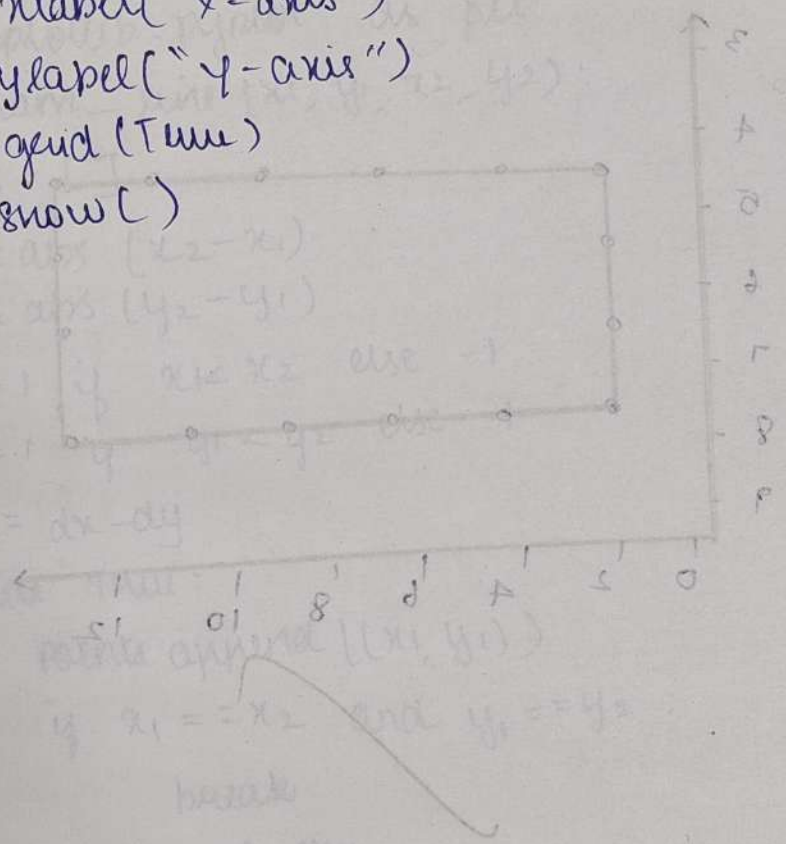


```
x_points.append(round(x))
y_points.append(round(y))
x+ = x_inc
y+ = y_inc
plt.plot(x_points, y_points, marker='o')
plt.title("DDA line drawing Algorithm")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.grid(True)
plt.show()
```



Result: Thus the implementation of DDA using Bresenham's Algorithm was done successfully.

```
x_points.append(round(x))
y_points.append(round(y))
x+ = x_inc
y+ = y_inc
plt.plot(x_points, y_points, marker='o')
plt.title("DDA line drawing Algorithm")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.grid(True)
plt.show()
```



Result: Thus the implementation of DDA using Bresenham's algorithm was done successfully.

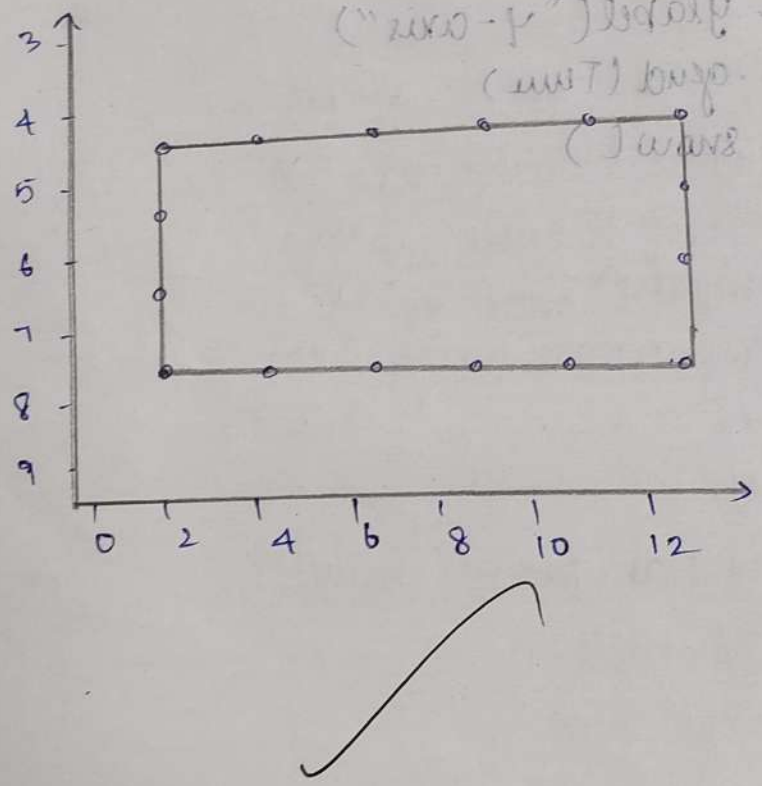
Exp: 6
M/8/26

Aim:

To dra
algorithm
(2,2), wid

ode:

import m
def brese
poi
dx
dy
st
st
cu
v



Exp: 6
11/8/26

Bresenham Rectangle Drawing Algorithm

Aim:

To draw a rectangle using bresenham algorithm with measurements top to left (2,2), width = 8, ht = 6.

code:

```
import matplotlib.pyplot as plt
def bresenham_line(x1, y1, x2, y2):
    points = []
    dx = abs(x2 - x1)
    dy = abs(y2 - y1)
    sx = 1 if x1 < x2 else -1
    sy = 1 if y1 < y2 else -1
    err = dx - dy
    while True:
        points.append((x1, y1))
        if x1 == x2 and y1 == y2:
            break
        e2 = 2 * err
        if e2 > -dy:
            err -= dy
            x1 += sx
        if e2 < dx:
            err += dx
            y1 += sy
    return points
```

```
x = 2
y = 2
width = 8
height = 5
sides = [ bresenham_line (x, y, x2, y),
bresenham_line (x2, y, x2, y2),
bresenham_line (x2, y2, x, y2),
bresenham_line (x, y2, x, y) ]
for side in sides : xs = [p[0] for p in side]
                    ys = [p[1] for p in side]
                    plt.plot (xs, ys, marker = 'o')

plt.xlim (0, 12)
plt.ylim (9, 0)
plt.grid (True)
plt.xlabel ("x")
plt.ylabel ("y")
plt.title ("Rectangle")
plt.show ()
```

Result :

Thus rectangle was created using the Bresenham algorithm, with the given measurements.

Exp:-1
11/8/26

Traffic Signal

Aim:

Draw a Traffic signal using circle and fill appropriate colours.

code:

```
import matplotlib.pyplot as plt
from matplotlib.patches import
Rectangle, Circle
fig, ax = plt.subplots()
ax.add_patch(Rectangle((2,2), 4, 10, color="black"))
ax.add_patch(Circle((4,10), 1, color="red"))
ax.add_patch(Circle((4,7), 1, color="yellow"))
ax.add_patch(Circle((4,4), 1, color="green"))
ax.add_patch(Rectangle((3.5,-2), 1, 4))
ax.set_xlim(0,8)
ax.set_ylim(-2,13)
ax.set_aspect("equal")
ax.axis("off")
plt.title("Traffic Signal")
plt.show()
```

Result:

Thus, a traffic signal is drawn using circle.

Exp:-7
11/8/26

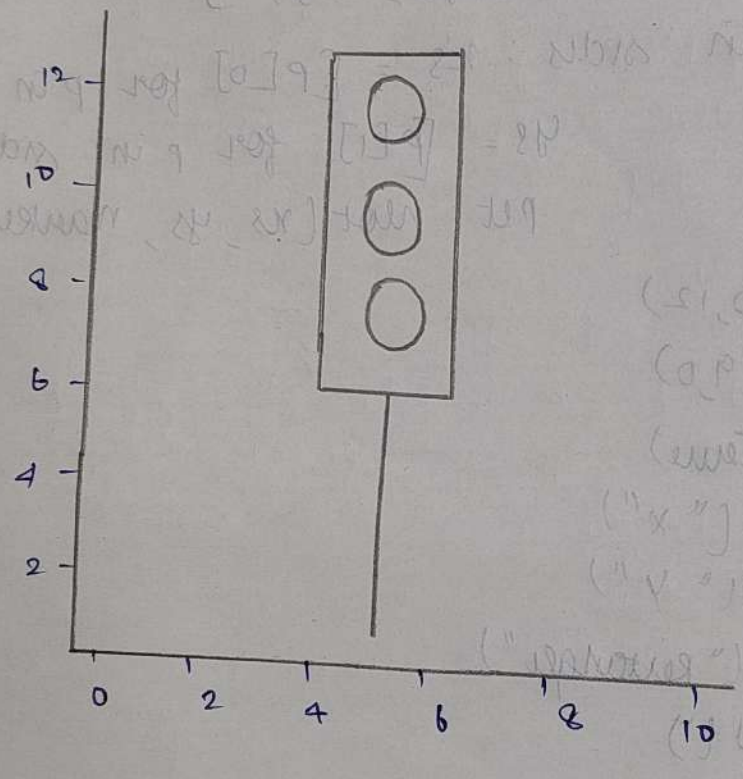
Aim:

Draw a

fill appro

code:

import matp
from matp
Rectangle,
fig, ax =
ax.add_
ax.add_
ax.add_p
ax.add_p
ax.add_
ax.set_
ax.set_
ax.set_
ax.axis
plt.titl
plt.show



Result:

usi

Exp: 8
11/8/26

Smiling Face

Aim:

To draw a smiling face using circle, and fill colors with eyes, nose & mouth

code:

```
import matplotlib.pyplot as plt
from matplotlib.patches import
Circle, Arc
fig, ax = plt.subplots()
ax.add_patch(Circle((5,5),4, color="yellow"))
ax.add_patch(Circle((3.5,6),0.5, color="black"))
ax.add_patch(Circle((6.5,6),0.5, color="black"))
ax.add_patch(Circle((5,4.2),1,2.5, angle=0,
theta2=340, color="black", lw=4))
ax.set_xlim(0,10)
ax.set_ylim(0,10)
ax.set_aspect("equal")
ax.axis("off")
plt.title("Smiling Face")
plt.show()
```

Result:

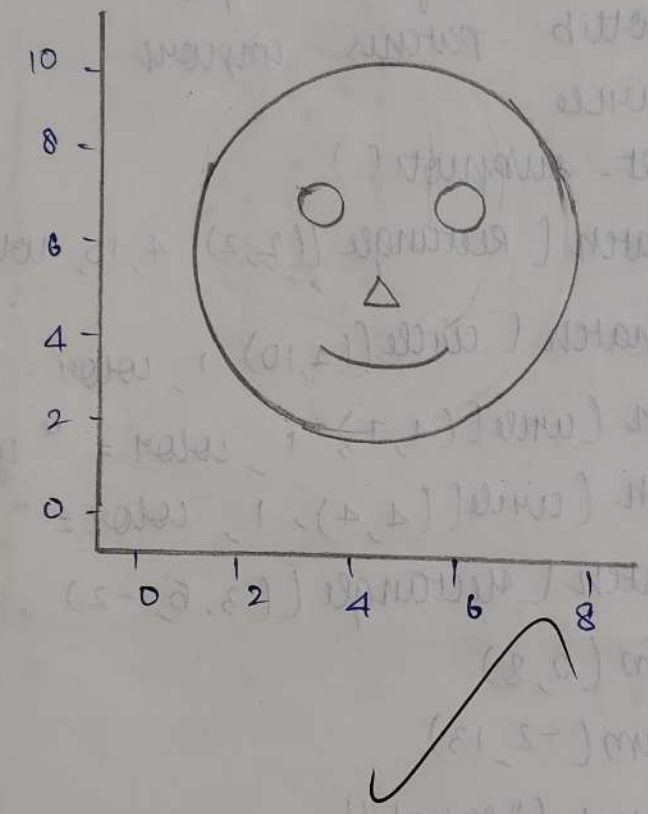
Thus, a smiling face was created using circle with eyes, nose and mouth.

Exp: 8
11/8/26

Aim:
To dr
and fill

code:

import m
from m
circle, A
fig, ax =
ax.add
ax.add
ax.add
ax.add
theta2 =
ax.se
ax.s
ax.se
ax.
plt.t
plt.s



Result
using

EXP NO: 9
17/8/26

Olympic Logo

Aim:

To draw a olympic logo using ellipse.

code:

```
import matplotlib.pyplot as plt
from matplotlib.patches import
Ellipse
fig, ax = plt.subplots()
rings = [(3,5,"blue"), (5,5,"black"),
(7,5,"red"), (4,3.2,"yellow"), (6,3.2,"green")]
for x,y color in rings:
    ax.add_patch(Ellipse((x,y), 3, 2, fill=False,
color=color, linewidth=5))
ax.set_xlim(1,9)
ax.set_ylim(1,7)
ax.set_aspect("equal")
ax.axis("off")
plt.title("Olympic Logo using Ellipse")
plt.show()
```

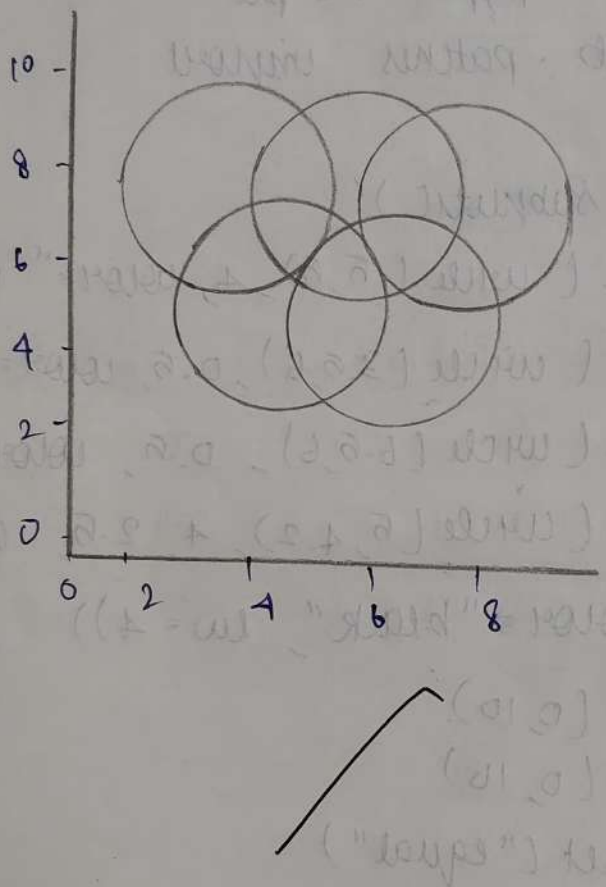
Result:

Thus, the olympic logo was created using ellipse.

EXP NO: 9
17/8/26

Aim:

Code:



EXP NO: 10
17/8/26. Butterfly

Aim: To create a colourful butterfly with eyes and antenna.

code:

```
import matplotlib.pyplot as plt
from matplotlib.patches import
Ellipse, circle
fig, ax = plt.subplots()
for x, c in [(3, "hotpink"), (7, "hotpink"),
(3, "plum"), (7, "plum")]:
    ax.add_patch(Ellipse((x, 6 if c="hotpink"
                          else 3.5),
    c=="hotpink" else 3.4 if
    c=="hotpink" else 2.5, ec="black)
    for x in [2.5, 7.5]:
    ax.add_patch(circle((x, 6.5), 6, color=
"turquoise", ec="black", lw=2))
    ax.add_patch(circle((x, 3.5), 4, color=
"yellow", ec="black", lw=2))
    ax.add_patch(Ellipse((5, 5), 1, 5, color=
"yellow", ec="black", lw=3))
```

EXP NO: 10
17/8/26.

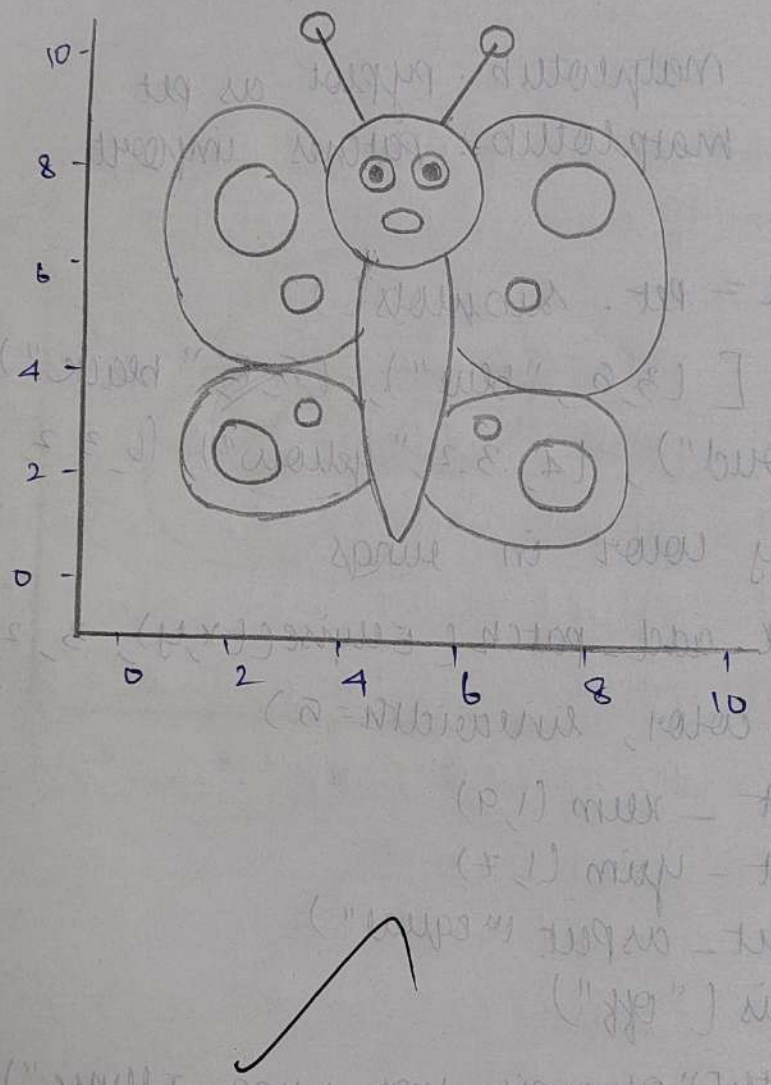
Aim: To

eyes and

code:

```
import
from ma
Ellipse
fig, ax =
for x,
(3, "blu
ax.

c=="
c==
for
ax.
"turq
ax.
"yello
ax
"yello
```



```
ax.add_patch (circle (5,7.5), color = "yellow",
    ec = "black", lw = 3)
for x in [4.6, 5.4]:
    ax.add_patch (circle [x, 7.7), 3, color = "white"))
    ax.add_patch (circle [x, 7.7), 12, color = "black"))
    ax.add_patch (circle [x, 5), 6, color = "red"))
    ax.plot ([4.6, 3.5], [8.2, 9.2], "k")
    ax.plot [[5.4, 6.5], [8.2, 9.2], "k"]
    ax.set (xlim = (0,10), ylim = (1,10))
    ax.set (aspect ("equal"))
    ax.axis ("off")
    plt.show()
```

Result:

Thus a butterfly is created using circle with eyes and antenna.

Exp: 11
15/9/22

Implement the scan conversion of polygon and use floodfill algorithm to fill areas in a graphics window.

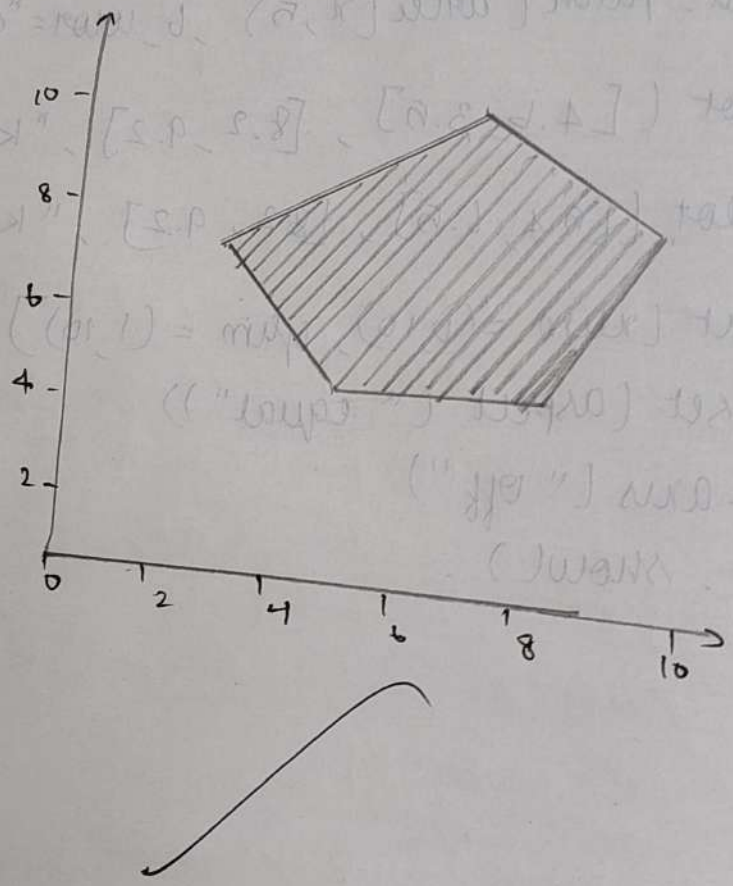
Aim: To Implement scan conversion of a polygon and fill its area using flood fill alg.

Program:

```
import matplotlib.pyplot as plt
from collections import deque
def Bresenham(x1, y1, x2, y2):
    points = []
    dx = abs(x2 - x1)
    dy = abs(y2 - y1)
    sx = 1 if x1 < x2 else -1
    sy = 1 if y1 < y2 else -1
    err = dx - dy
    while True:
        points.append((x1, y1))
        if x1 == x2 and y1 == y2:
            break
        e = 2 * err
        if e > -dy:
            err -= dy
            x1 += sx
```

Exp: 11
15/9/22

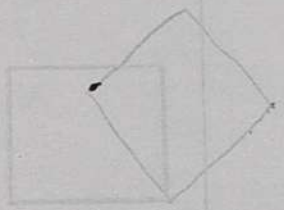
Scan conversion of polygon by floodfill.



Aim:
To
and fi
Program
import
from
def BR

```
        if e < dx:
            e += dx
            y1 += sy
    return points
polygon = [(2,2), (7,2), (8,5), (5,7), (1,5)]
boundary = set()
for i in range(len(polygon)):
    x1, y1 = polygon[i]
    x2, y2 = polygon[(i+1) % len(polygon)]
    boundary.update(bresenham(x1, y1, x2, y2))
def flood_fill(seed):
    queue = deque([seed])
    filled = set()
    while queue:
        x, y = queue.popleft()
        if (x, y) in boundary:
            continue
        queue.append((x+1, y))
        queue.append((x-1, y))
        queue.append((x, y+1))
        queue.append((x, y-1))
return filled
filled = flood_fill((4,4))
for x, y in filled:
    plt.fill([x, x+1, x+1, x], [y, y, y+1, y+1])
```

```
bx, by = zip (*boundary)
plt.scatter (bx, by, s=30)
plt.xlim (0,9)
plt.ylim (0,9)
plt.xticks ( range(10))
plt.yticks ( range(10))
plt.grid()
plt.show().
```



22/9/26

Result:

Thus the polygon was scan converted using Bresenham's and its enclosed area is filled using flood fill algo.

14/9/26
Exp: 12

Basic 2D Transformations

Aim:

To draw a basic shape using Bresenham's line drawing alg and apply Translation, Rotation and scaling Transformation.

Code:

```
import matplotlib.pyplot as plt
import math
def bresenham(x1, y1, x2, y2):
    points = []
    dx = abs(x2 - x1)
    dy = abs(y2 - y1)
    sx = 1 if x1 < x2 else -1
    sy = 1 if y1 < y2 else -1

    err = dx - dy
    while True:
        points.append((x1, y1))
        if x1 == x2 and y1 == y2
            break
        e = 2 * err
        if e > -dy:
            err -= dy
            x1 += sx
        if e < dx:
            err += dx
            y1 += sy
```

4/9/26
Exp: 12

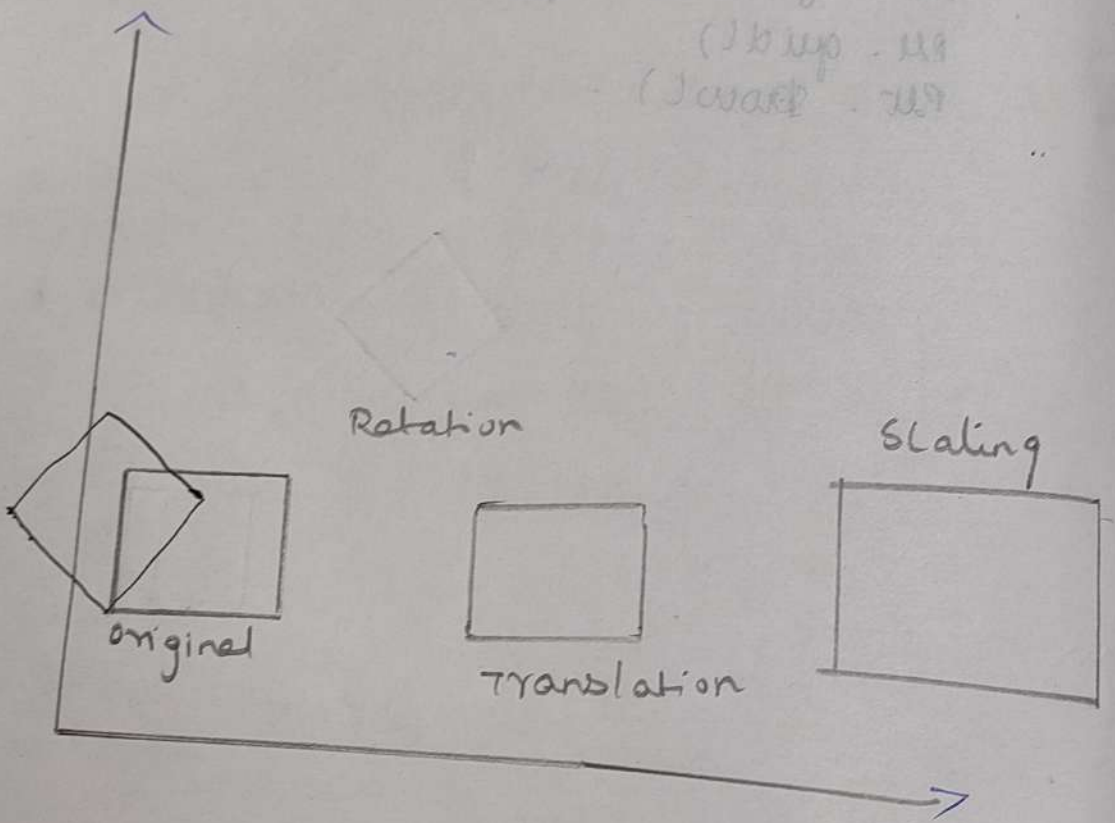
Aim:

To dra
Line Drawi
Rotation

Code:

import ma
import m
def bresen
points
dx = ab
dy = a
ix = 1
iy = 1
x4 =
while

2D Transformation



```
return points

def draw_polygon(polygon, label):
    for i in range(len(polygon)):
        x1, y1 = polygon[i]
        x2, y2 = polygon[(i+1) % len(polygon)]
        points = bresenham(x1, y1, x2, y2)
        x, y = zip(*points)
        plt.scatter(x, y, s=15)
    plt.plot([], [], label=label)

polygon = [(2,2), (5,2), (4,5)]
translated = [(x+5, y+1) for x, y in polygon]
angle = math.radians(45)
rotated = math.radians[]
for x, y in polygon:
    xr = round(x * math.cos(angle))
    yr = round(x * math.sin(angle))
    rotated.append((xr, yr))
scaled = [(x*2, y*2) for x, y in polygon]
draw_polygon(polygon, "Original")
draw_polygon(translated, "Translation")
draw_polygon(rotated, "Rotation")
draw_polygon(scaled, "scaled")
plt.grid()
plt.axis(equal)
plt.legend()
plt.show()
```

Result :- 22/9/22

Thus, a polygon was successfully drawn using Bresenham alg and Translation, Rotation, Scaling transformations were applied.

29/9/21
Exp. 13

Polygon clipping

Aim:

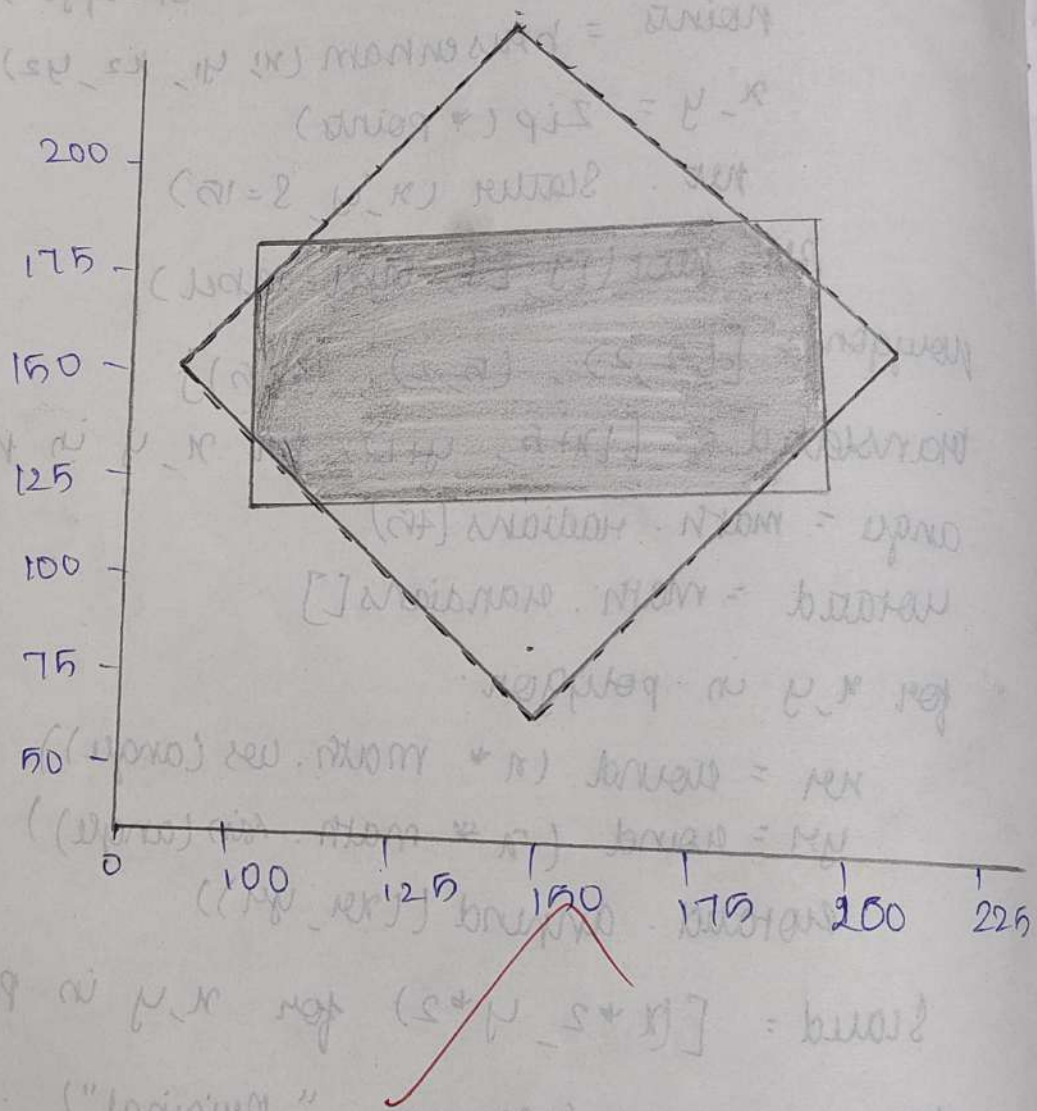
To implement polygon clipping using the Sutherland - Hodgman polygon clipping algorithm & visualise the clipped polygon using Python, shapely and matplotlib.

Code:

```
! pip install -q shapely matplotlib
import matplotlib.pyplot as plt
from shapely.geometry import Polygon
poly_subj = Polygon([(100,150), (200, 250),
                     (300,150), (200, 50)])
poly_clip = Polygon([(120, 80), (120, 200), (280, 200),
                     (280, 80)])
poly_result = poly_subj.intersection(poly_clip)
fig, ax = plt.subplots(figsize=(6,6))
ax.plot(*poly_subj.exterior.xy, 'b--',
        label='Original Polygon')
ax.plot(*poly_clip.exterior.xy, 'g:',
        label='Clipping window')
ax.plot(*poly_result.exterior.xy, 'r',
        label='Clipped Result')
```

29/9/21
Exp: 13

Aim:



```
ax.set_title('Sutherland Hodgman')
ax.legend()
ax.grid(True)
plt.show()
```

Result:

The polygon was successfully clipped according to the specified window and the clipped vertices were displayed.

29/9/21
Exp: 14

cube 3D Transformation.

Aim:

To implement & visualize 3D transformations.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d
                    import Poly3DCollection

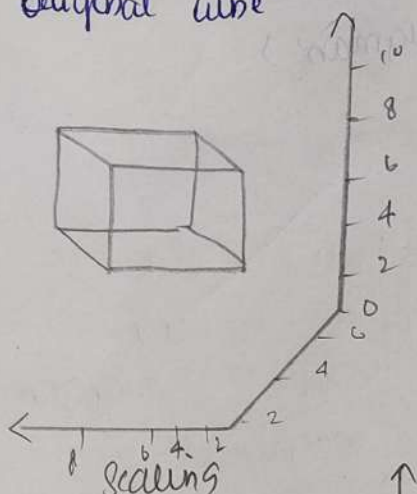
V = np.array([ [0,0,0], [1,0,0], [0,1,0] [0,0,1],
               [1,1,0] [1,0,1] [0,1,1] [1,1,1] ])

edges = [ (0,1) (0,2) (0,3) (1,4) (1,5) (2,4),
          (2,6) (3,5) (3,6) (4,7) (5,7) (6,7)]

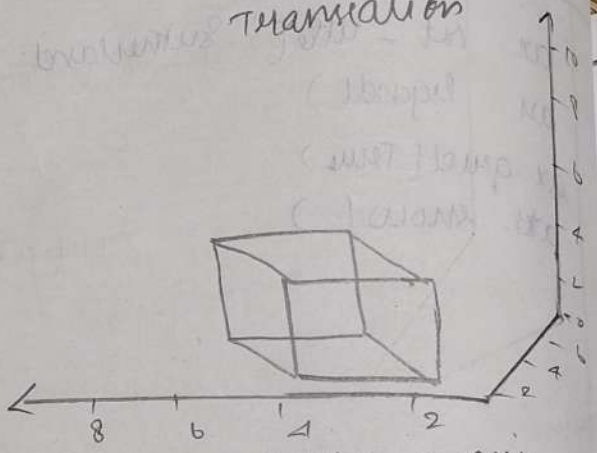
def draw (ax, P, color, label):
    for i,j in edges:
        ax.plot (*zip(P[i], P[j]), color =
                 color)
    ax.scatter (P[:0], P[:,1], P[:,2],
    color = color, label = label)

def transform (matrix, title, color):
    H = np.c_[V, np.ones(8)]
    P = (matrix @ H.T).T[:,:3]
    fig = plt.figure()
```

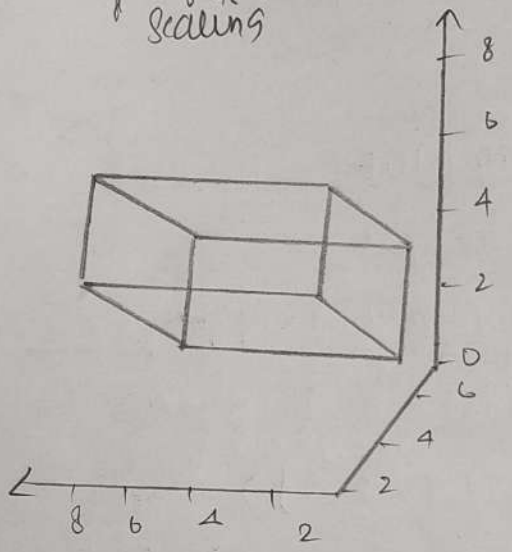
Original cube



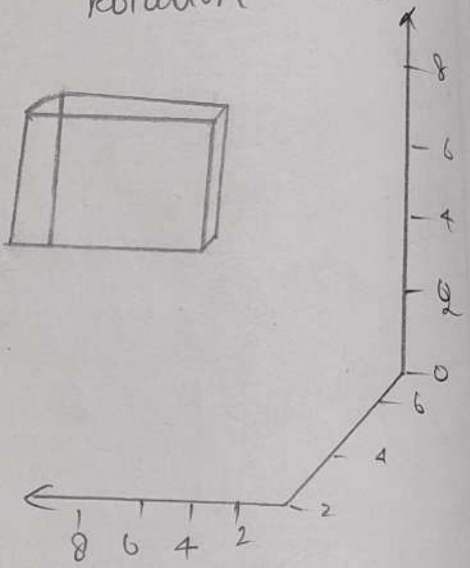
Translation



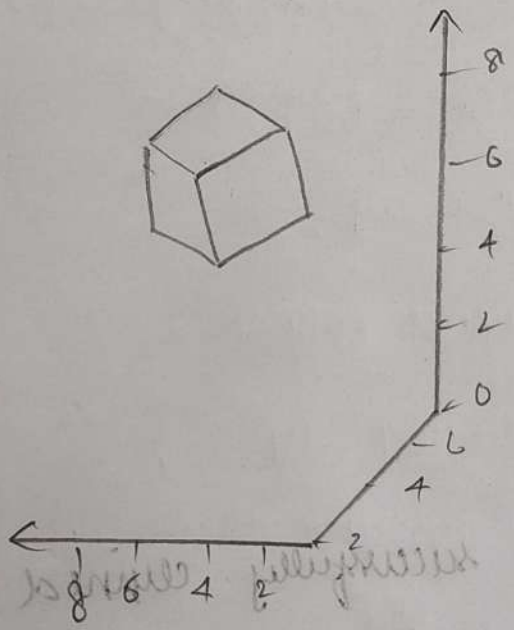
Scaling



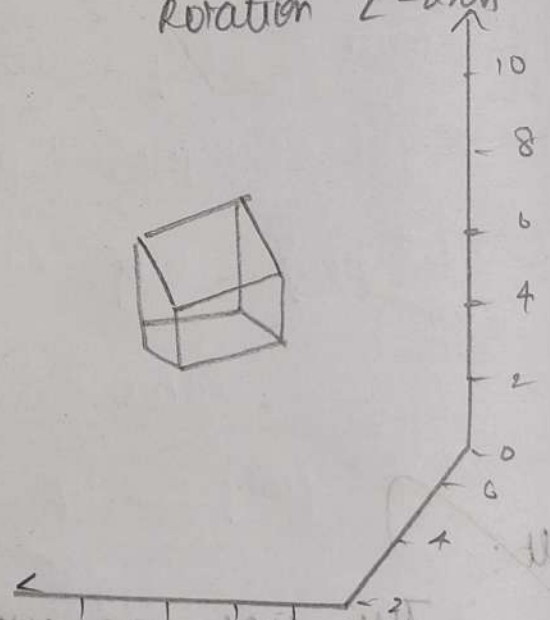
Rotation x-axis



Rotation y-axis



Rotation Z-axis



29/9/21
Exp: 14

Aim:

Code:

import

import

from

V = np

edge

def

def

```
ax = fig.add_subplot(111, projection='3d')
draw(ax, V, 'blue', 'original')
draw(ax, P, color, 'Transformed')
ax.set_title(title)
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")
ax.legend()
T = np.array([1,0,0,2] [0,1,0,2]
             [0,0,1,2] [0,0,0,1])
a = np.pi/4
R = np.array([np.cos(a), -np.sin(a),
              0,0],
             [np.sin(a), np.cos(a), 0,0],
             [0,0,1,0] [0,0,0,1])
transform(R, "Rotation", "orange")
S = np.diag([1.5, 0.5, 2, 1])
transform(S, "Scaling", "red")
```

Result:

Thus the implementation of cube transformation is done successfully.

6/10/26
Exp: 15

Cone 3D Transformation

Aim:

To implement and visualize cone 3D transformations.

code:

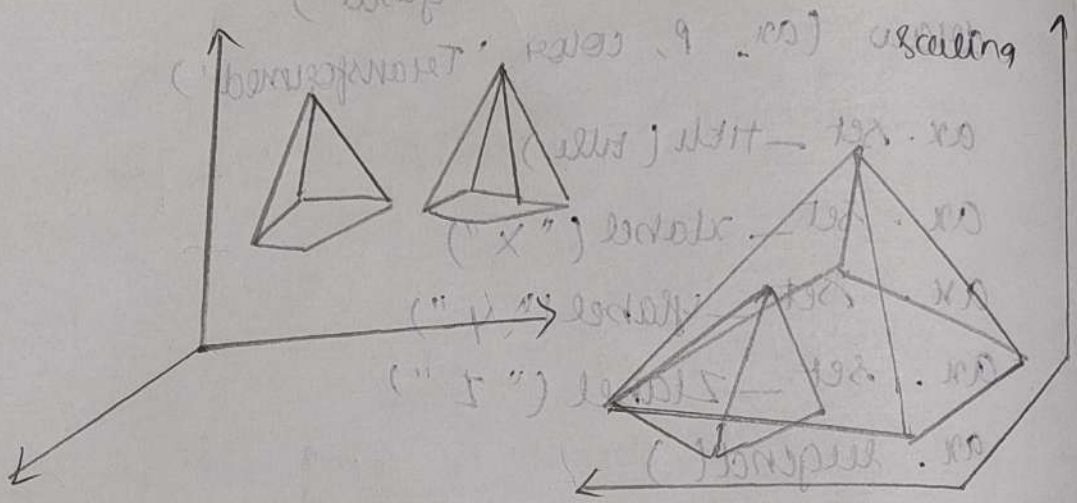
```
import numpy as np
import matplotlib.pyplot as plt
V= np.array [[0,0,0] [1,0,0] [1,1,0] [0,1,0]
             [5,5,1]]
edges = [(0,1)(1,2)(2,3)(3,0)(0,4)(1,4)(2,4)]
def draw (ax, P, label)
    for i, j in edges:
        ax.plot (*zip([P[i], P[j]),
                 color=())
    ax.scatter (P[:,0] [P:1] [P:2])
def transform (M, title, color):
    h = np.c_ [V, np.ones (5)]
    P = (M @ h.T).T[:, :3]
    fig = plt.figure()
    ax = fig.add_subplot (111, projection
                          = '3d')
```

6/10/26
Exp: 15

Aim

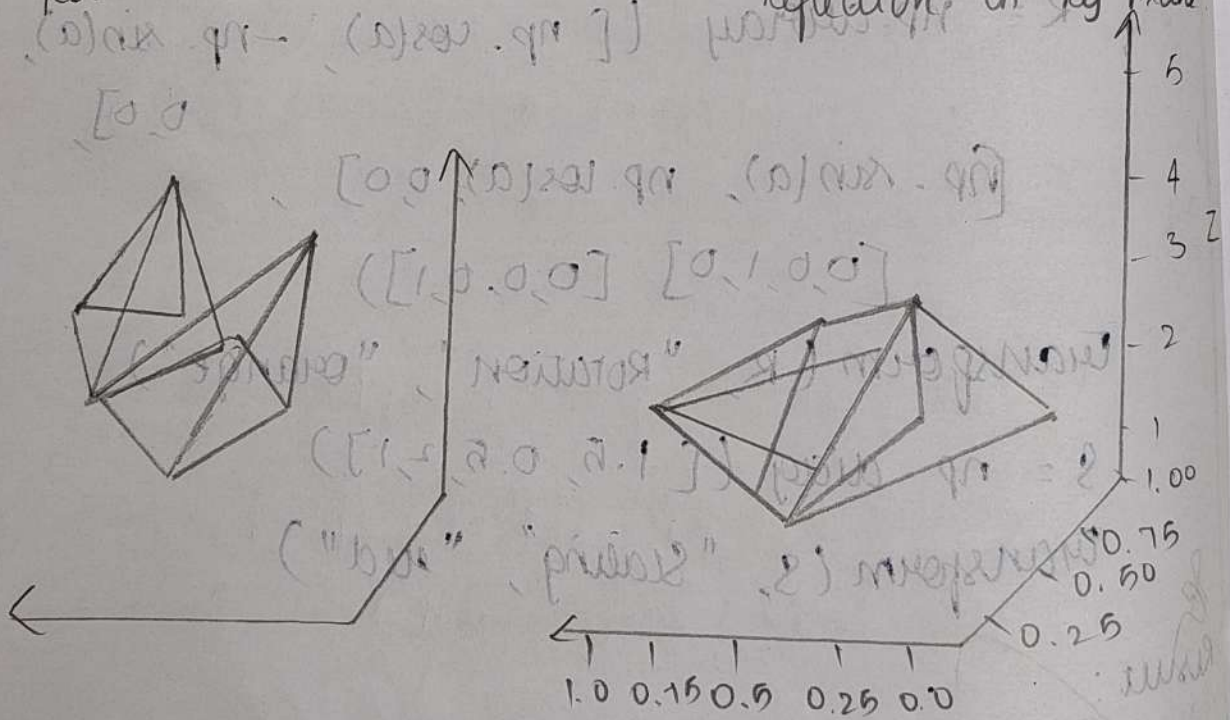
- 3D

code



Rotation alt - z axis

Reflection in xy plane



```
draw(ax, V, 'blue', 'original')
draw(ax, P, color, 'Transformed')
ax.set_title(title)
plt.show()
T = np.array([[1,0,0,2],[0,1,0,2],
              [0,0,1,2],[0,0,0,1]])
Transform(T, "Pyramid Translation",
                          "green")
a = np.pi/4
R = np.array([[np.cos(a), -np.sin(a),
    0],
    [np.sin(a), np.cos(a),
     0,0,1,0],[0,0,0,1]])
Transform(R, "Pyramid")
S = np.diag([1.5, 1.5, 1])
Transform("Pyramid Scaling")
```

Result: Thus the 3D transformations were done successfully.

Exp: 16
6/10/25

Create a Multimedia Application that integrates Images, Sound & video.

Aim:

To develop a simple multimedia application that integrates.

* Images
* video
* Audio

Code:

```
import tkinter as tk
from PIL import Image, ImageTk
from playsound import playsound
import cv2
import threading
def audio():
    playsound("sample_audio")

def video():
    cap = cv2.VideoCapture("sample_video")
    while True:
        ret, frame = cap.read()
        if not ret: break
        cv2.imshow("video")
    cap.release()
```

Exp: 16
6/10/25

Aim:

To

that in

*

*

*

Code:

impor

from

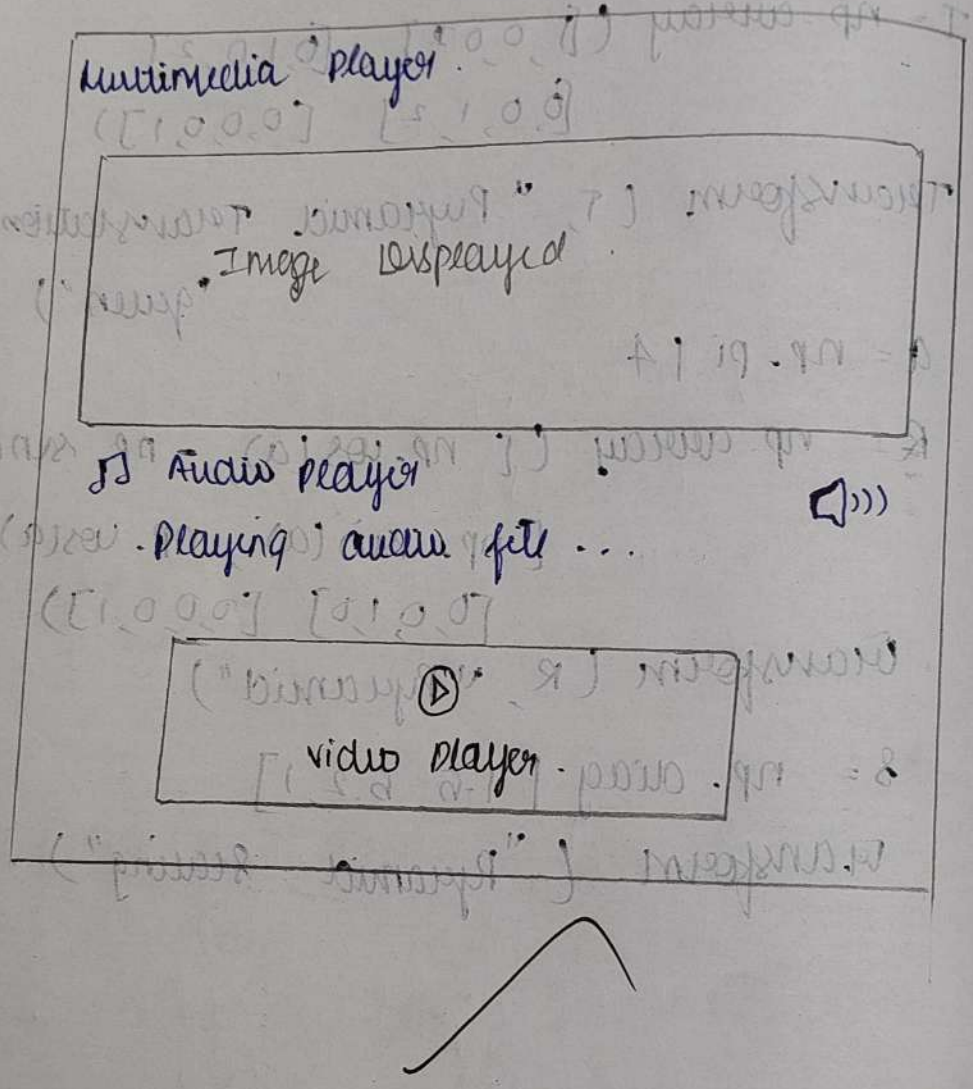
from n

impor

impor

def au

def vi



```
def Image():
    img = Image.open("sample_image")
    photo = ImageTk.PhotoImage(img)
    panel.config()
    panel.image = photo

win = tk.Tk()
win.title("Multimedia App")
panel = tk.Label(win)
panel.pack()
tk.Button(win, text = "Show Image",
          command = image).pack()
tk.Button(win, text = "Play Audio",
 command = lambda : threading.Thread())
tk.Button(win, text = "play video",
  command = lambda : threading.Thread()).
win.mainloop().
```

Result:

A multimedia application was successfully created that integrates image, audio, and video.

Exp: 17
6/10/26

Capture video/audio from webcam or microphone and display on Multimedia Interface.

Aim:

To develop a program that captures video, audio etc.

coding:

```
import cv2
import sounddevice as sd
import tkinter as tk
import threading

def audio():
    fs = 44100
    data = sd.rec(5*fs, samplerate = fs, channels=2)
    sd.wait()
    write("recorded_audio.wav", fs, data)

def video():
    cap = cv2.VideoCapture(0)
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        cv2.imshow("Webcam", frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
```

output :-

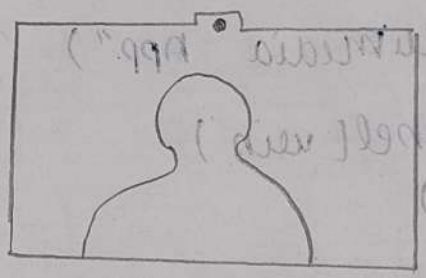
1) Main GUI window :-

capture Audio and video

Record Audio (5 sec)

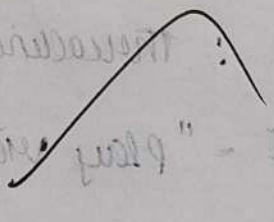
Start Webcam

2) webcam output :-



3) Audio output :

recorded_audio.wav



Exp: 17
6/10/26

Aim:

To

audio

coding:

import

import

import

import

def audio

fs

du

sd

wa

def vid

cap

wri

```
cap.release()
cv2.destroyAllWindows()

root = tk.Tk()
root.title("Audio and Video Capture")
tk.Button(root, text="Record Audio",
          command=lambda: threading.Thread
          (target=audio).start()).pack(padx=10)
tk.Button(root, text="Start Webcam",
          command=lambda: threading.Thread
          (target=video).start()).pack(pad=10)
root.mainloop()
```

Result:

Live video and audio were successfully captured using webcam and microphone & displayed/saved through a multimedia interface.

Exp: 18
11/10/25

Create an application to send and receive multimedia messages (Text, Image, Audio)

Aim:

To develop a basic chat-like application that allows:

Sending and receiving text, images and audio using local file selection and simulated messaging UI.

code:

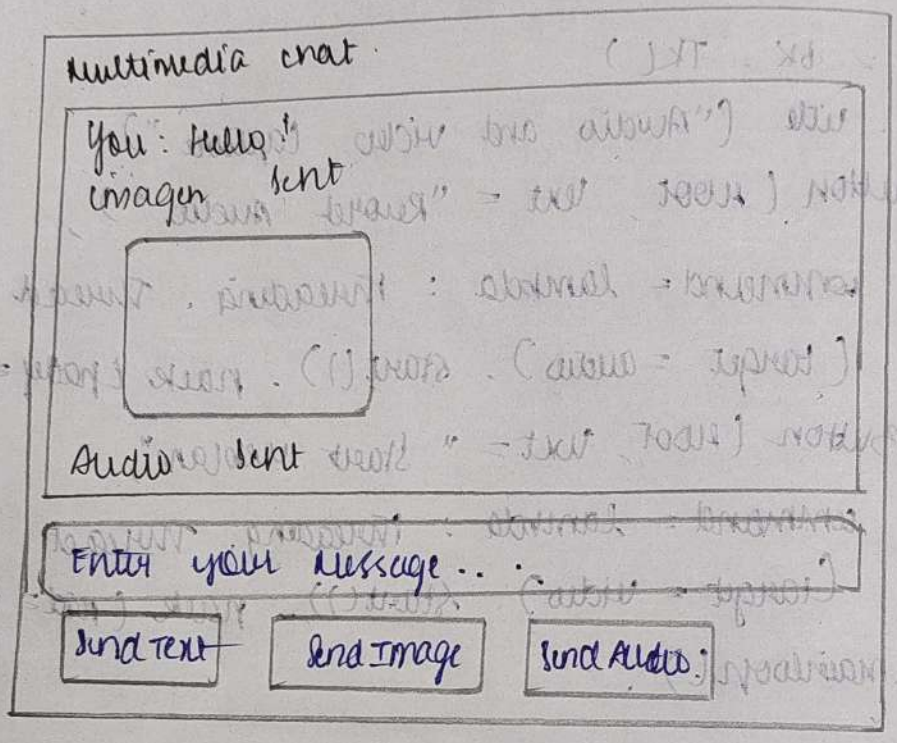
```
import tkinter as tk
from tkinter import filedialog
from PIL import Image, ImageTk
from playsound import playsound
import threading

def text():
    chat.insert(tk.END, "You: " + entry.get() + "\n")
    entry.delete(0, tk.END)

def image():
    path = filedialog.askopenfilename()
    if path:
        img = ImageTk.PhotoImage(Image.open(path))
        label.config(image=img)
        label.image = img
        chat.insert(tk.END, "Image sent\n")

def audio():
    path = filedialog.askopenfilename()
```

Output :-


Multimedia chat
You: Hello!
Image sent
Audio sent
Enter your message...
Send Text
Send Image
Send Audio

Exp:19
11/10/26

Create a 3D model using Blender and apply basic shading and texture.

Aim:

To create a simple 3D model using Blender.

Procedure:

1) Launch Blender
2) Model creation
   Use add mesh to insert shapes.
   combine mese to form house or similar structure.
   used Edit mode for shaping
3) Shading:
   Switch to the shading workspace
   Add new material for each object
   Adjust base color, specular.
4) Texture mapping:
   Load an External image
   Use UV editing to align the texture.
5) Lighting and camera:
   Add light sources
6) Render settings:
   choose cycles or Eevee renderer.
   Adjust resolution and sampling.
   click Render > Render Image to produce output.

Result: A 3D object was successfully modeled in Blender. Basic shading and texture mapping techniques were applied.

```
if path:
    chat.insert("Audio Sent")

label = tk.Label(root)
label.pack()
entry.pack()
tk.Button(root, text="Sent Text").pack()
tk.Button(root, text="Sent Image").pack()
tk.Button(root, text="Sent Audio").pack()
root.mainloop()
```

Result:

A multimedia messaging was successfully created with options to send text, image and audio content. The application simulated in local environment.

EXP-19
11/10/26

Aim:

To create a

Procedure:

1) Launch Blender
2) Model creation
   Use add mesh
   combine more
   used Edit
3) Shading:
   Switch to the
   Add new mat
   Adjust base
4) Texture mapping
   Load an E
   Use UV Edi
5) Lighting and
   Add light
6) Render settings
   choose cycles
   Adjust resol
   F12 Render

Result:

A 3D
Blender. Basic
techniques were